

# Six Months with AMD Strix Halo

Local AI, Open Source, and Hardware Reality

A Six Month Review of a Strix Halo Rig

*AMD Strix Halo | Local AI | Home Assistant | 3D Modeling | Open Source*

winmutt | June 2026

[github.com/winmutt](https://github.com/winmutt)

# What We're Covering

## Section 1: The Hardware Stack

- Power Consumption: Corsair 300 vs RTX 5090 vs Cloud
- The Hardware: AMD Ryzen AI Max+ 395 APU
- The Stack: ROCm + Lemonade Server
- Issue #1070: Core affinity across dual CCDs

## Section 2: Projects & Applications

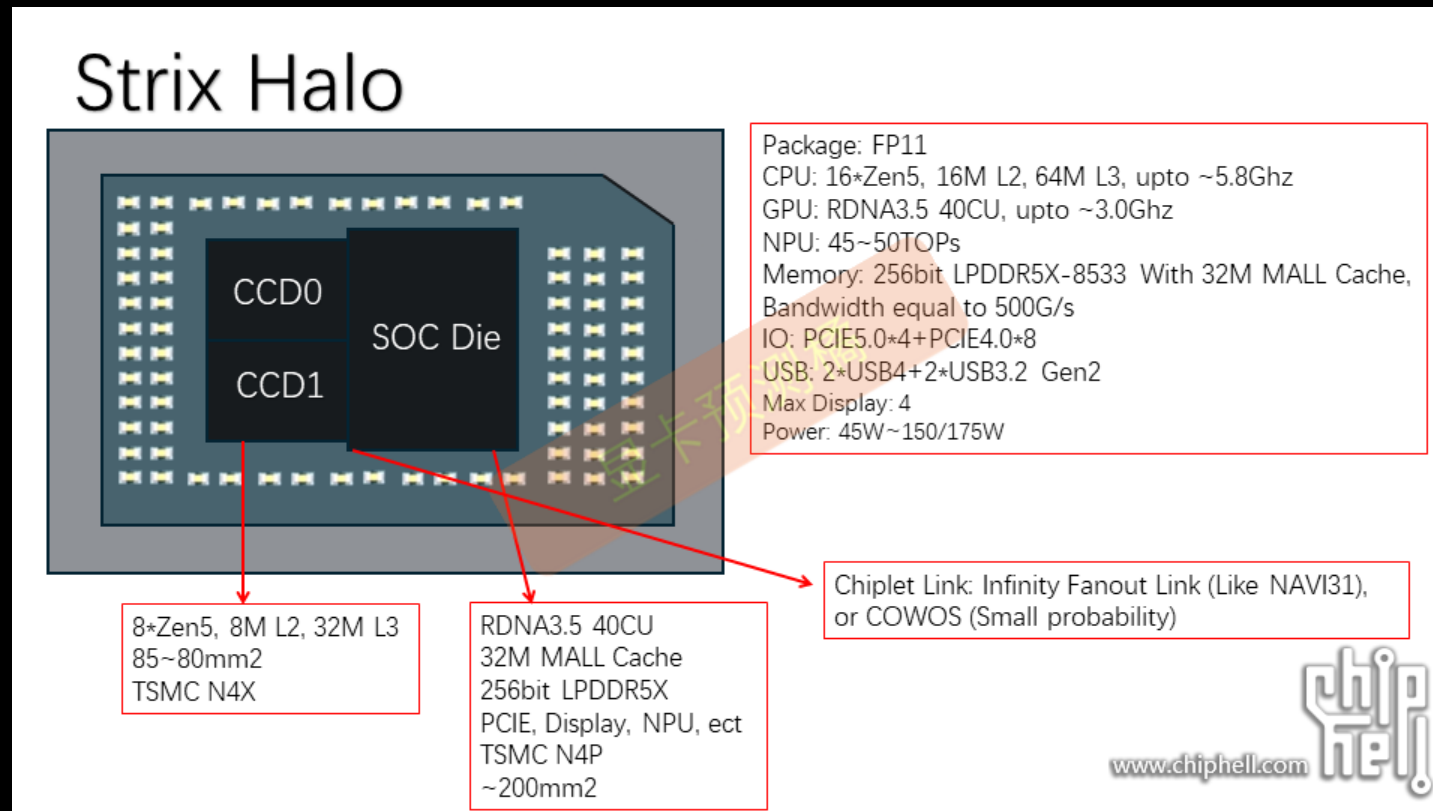
- Home Assistant: Cutting the Alexa cord
- 3D Modeling: AI to concrete signs
- AI Coding Editors: Opencode, Cline, Aider
- Project WWS: Remote workspace provisioning

## Section 3: Open Source Journey

- Lessons Learned: What worked, what didn't
- Key Insights: 8 takeaways from 6 months
- OSS Contributions: 3.5x increase
- 26 Years: From LKML 2000 to Strix Halo

# The Hardware: AMD Ryzen AI Max+ 395

## APU Die Architecture



## Physical Layout

- 16 cores (12 performance + 4 efficiency)
- 2x Core Complex Dies (CCD) with 32MB L3 cache each
- 768KB L1d + 512KB L1i + 16MB L2
- Integrated RDNA 3.5 GPU (40 Compute Units)

- 128GB LPDDR5X-8000 unified memory (soldered)
- Source: AMD Ryzen AI Max+ 395 teardown analysis (Chiphell, TechPowerUp)

## **AMD Radeon 680M GPU**

- RDNA 3.5 architecture, 40 Compute Units
- Peak performance: ~4.6 TFLOPS
- Shared memory with CPU (128GB unified)
- Supports ROCm for AI/ML workloads
- Source: AMD Ryzen AI Max+ 395 specifications

## **AMD XDNA 2 NPU**

- Neural Processing Unit for AI workloads
- Integrated into Ryzen AI Max+ 395 APU
- Supports FastFlowLM for efficient inference
- Power-efficient alternative to GPU processing

## **FastFlowLM Performance (June 2026):**

- 6784 tokens prefill in ~15 seconds
- ~455 tokens/sec on NPU
- Chunked processing: 4096 + 2688 tokens
- End-to-end (prefill to response): ~16.2 seconds
- Source: FastFlowLM logs (lemonade-server)

# Power Consumption Reality

## System Comparison

| System                       | Power Draw | Notes                                |
|------------------------------|------------|--------------------------------------|
| AMD Strix Halo (Corsair 300) | ~300W      | Single APU, 128GB unified memory     |
| RTX 4090 Setup               | ~600-800W  | GPU (450W TDP) + CPU + system        |
| RTX 5090 Setup               | ~800-1000W | GPU (600W TDP) + CPU + system        |
| Commercial AI Server         | 1500-3000W | Multi-GPU (A100/H100: 400-700W each) |

Strix Halo: ~300W vs RTX 5090: ~800-1000W vs Cloud Servers: 1500-3000W  
Source: Corsair specs, NVIDIA TDP ratings, TechPowerUp benchmarks

# The Hardware Investment

## Black Friday 2025 Deal

- Corsair 300 AI Workstation: **\$1800** (128GB Strix Halo system)
- Sold NVIDIA RTX 3060 rig to fund the purchase
- Today's equivalent: ~\$3000+ (RTX 4090 configurations)

**The Verdict:** "The Strix Halo was a bargain at \$1800 for what it delivers - today you'd pay 2x that for equivalent performance with discrete GPU"

## Feat #1070: [enhancement] Core affinity when running multiple models.

### Filed

February 8, 2026

### Repository

[github.com/lemonade-sdk/lemonade/issues/1070](https://github.com/lemonade-sdk/lemonade/issues/1070)

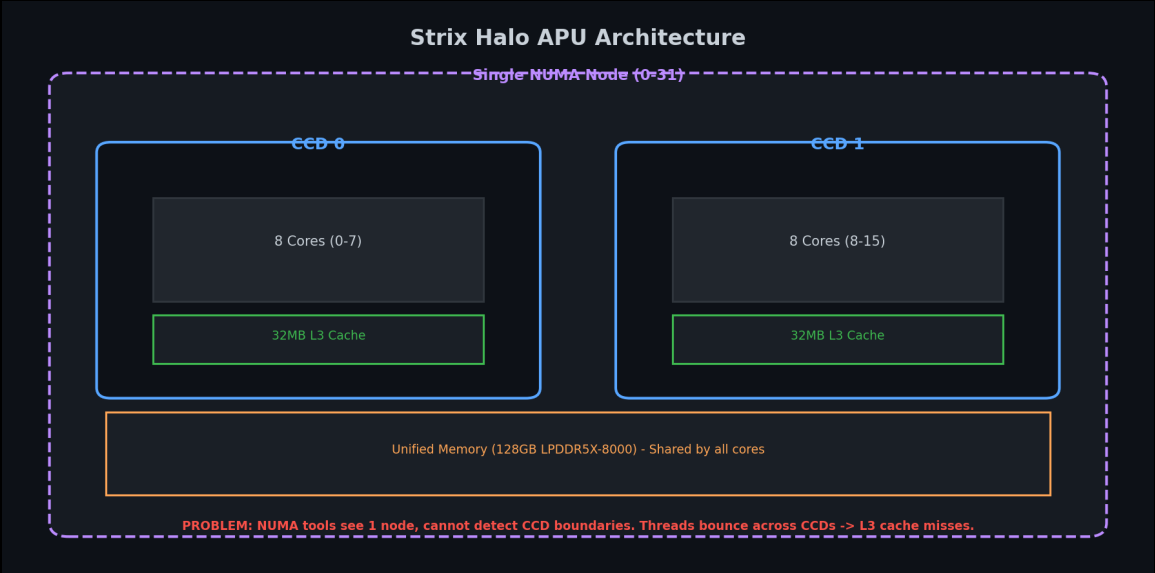
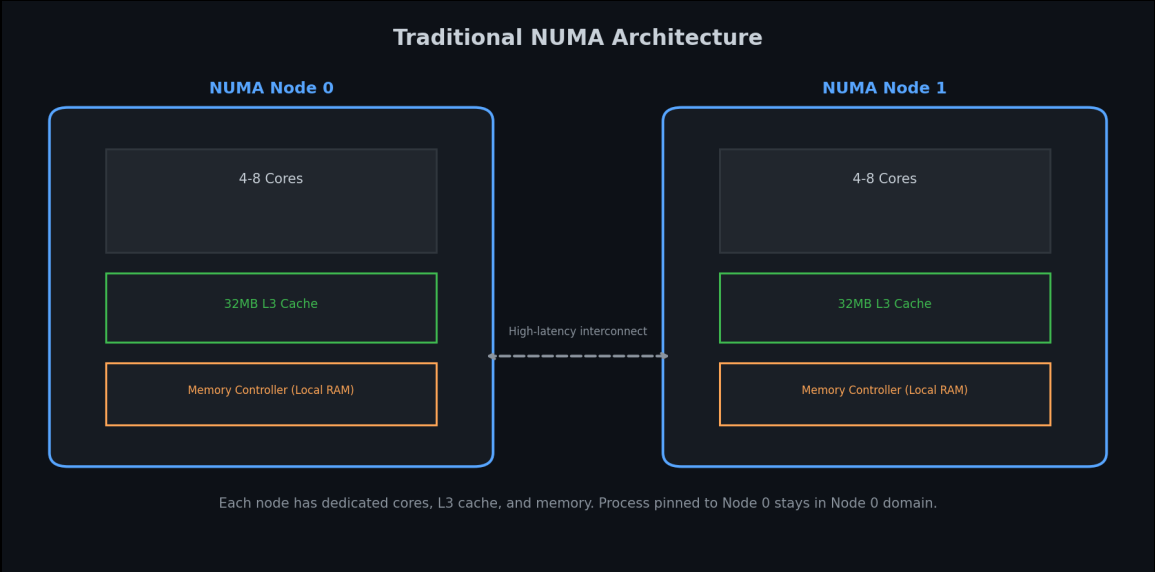
### Status

Open

### Hardware Reality

- One processor, but 2x core dies (CCDs) with separate 32MB L3 caches
- Traditional NUMA: 2 nodes, each with own cores + L3 cache
- Strix Halo: Single node, but 2 CCDs with separate L3 caches

# Traditional NUMA vs. Strix Halo APU





## The Problem: NUMA Tools Can't See CCD Boundaries

- Traditional NUMA tools (numactl, NUMATopologyFilter) detect only 1 node
- Cannot see CCD boundaries automatically
- Manual core pinning required to keep workloads on same CCD
- Running multiple LLMs: threads bounce across CCDs → L3 cache thrashing

**Result:** Cross-CCD traffic → slower inference, higher latency

## The Solution (Proposed)

Manual core pinning (like Red Hat's `vcpu_pin_set` approach):

- Reserve cores per CCD, pin llama-server accordingly
- Pin based on number of models loaded and CCD boundaries
- Prevent cache-crossing penalties

**Result:** Each model stays in its own L3 cache domain → no cross-CCD traffic

Evidence: Threads Bouncing Across CCDs

| ps Output: Threads Bouncing Across CCDs |          |         |                        |
|-----------------------------------------|----------|---------|------------------------|
| PID                                     | LAST_CPU | AVG_MHZ | COMMAND                |
| 56840                                   | 6        | 55.7    | llama-server (Model 1) |
| 56840                                   | 20       | 54.2    | llama-server (Model 1) |
| 59798                                   | 0        | 51.2    | llama-server (Model 2) |
| 59798                                   | 2        | 47.5    | llama-server (Model 2) |
| -----                                   |          |         |                        |
| 56840                                   | 22       | 55.7    | llama-server (Model 1) |
| 56840                                   | 4        | 54.2    | llama-server (Model 1) |
| 59798                                   | 16       | 51.2    | llama-server (Model 2) |
| 59798                                   | 18       | 47.5    | llama-server (Model 2) |

Notice: LAST\_CPU jumps across CCD boundaries

CCD0: cores 0-7, 16-23 | CCD1: cores 8-15, 24-31

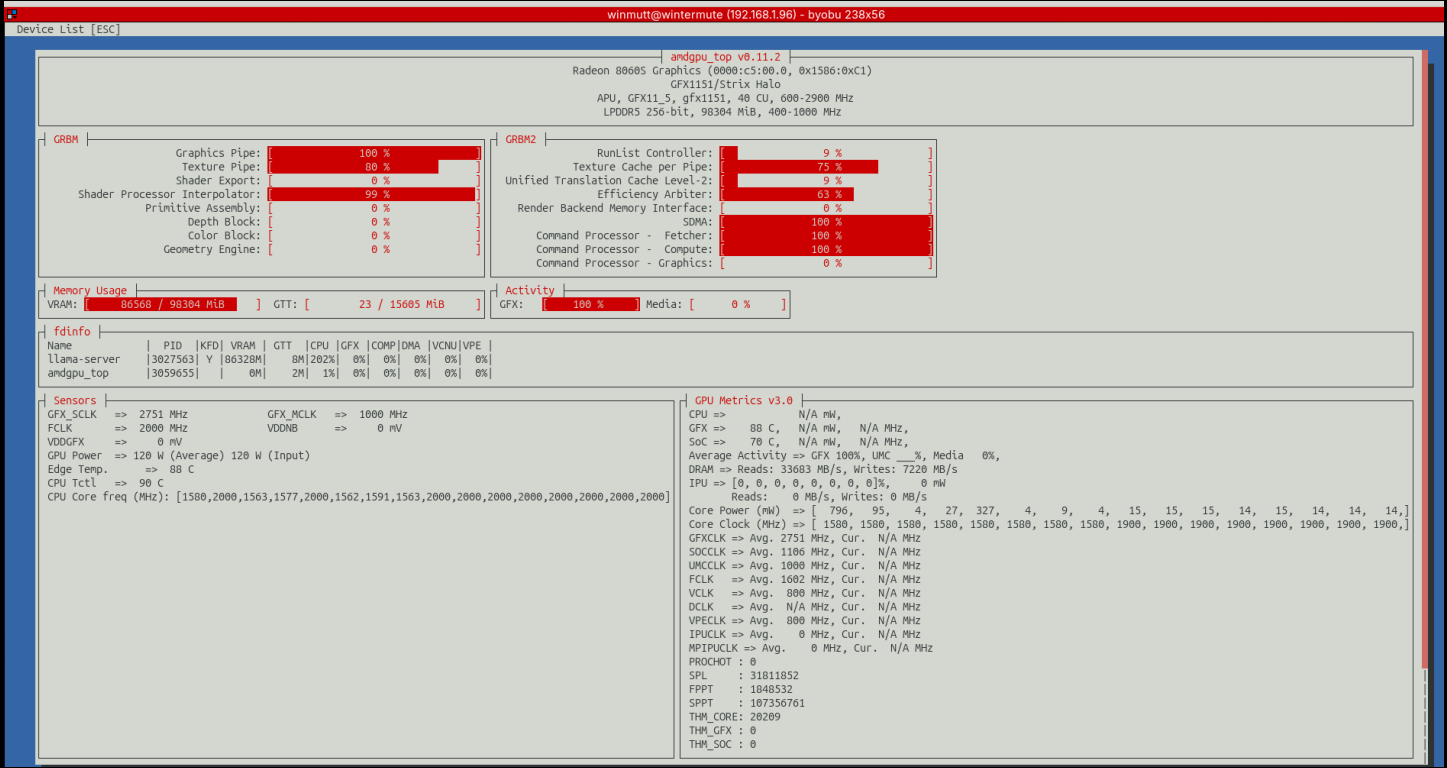
# Hardware Topology: CCD Boundaries (lscpu)

| lscpu Output: CCD Topology |      |      |    |        |         |
|----------------------------|------|------|----|--------|---------|
| CPU                        | NODE | CORE | L3 | MAXMHZ | AVG_MHZ |
| 0                          | 0    | 0    | 0  | 5187.0 | 4937.8  |
| 1                          | 0    | 1    | 0  | 5187.0 | 2000.0  |
| 2                          | 0    | 2    | 0  | 5187.0 | 4909.0  |
| 3                          | 0    | 3    | 0  | 5187.0 | 4904.9  |
| 4                          | 0    | 4    | 0  | 5187.0 | 4889.9  |
| 5                          | 0    | 5    | 0  | 5187.0 | 4845.2  |
| 6                          | 0    | 6    | 0  | 5187.0 | 4882.0  |
| 7                          | 0    | 7    | 0  | 5187.0 | 4950.4  |
| -----                      |      |      |    |        |         |
| 8                          | 0    | 8    | 1  | 5187.0 | 2000.0  |
| 9                          | 0    | 9    | 1  | 5187.0 | 2000.0  |
| 10                         | 0    | 10   | 1  | 5187.0 | 2000.0  |
| 11                         | 0    | 11   | 1  | 5187.0 | 2425.1  |
| 12                         | 0    | 12   | 1  | 5187.0 | 3056.8  |
| 13                         | 0    | 13   | 1  | 5187.0 | 4661.1  |
| 14                         | 0    | 14   | 1  | 5187.0 | 2000.0  |
| 15                         | 0    | 15   | 1  | 5187.0 | 2000.0  |

L3 Cache + AVG\_MHZ: Shows load shifting across CCD boundaries

NUMA tools see: 1 node (0-31) | Reality: 2 CCDs with separate L3 caches

Evidence: amdgpu\_top



# Cutting the Alexa Cord

## LineageOS on Echo

**December 2025**

Started exploring XDA Forums for Echo device hacking

### The Process:

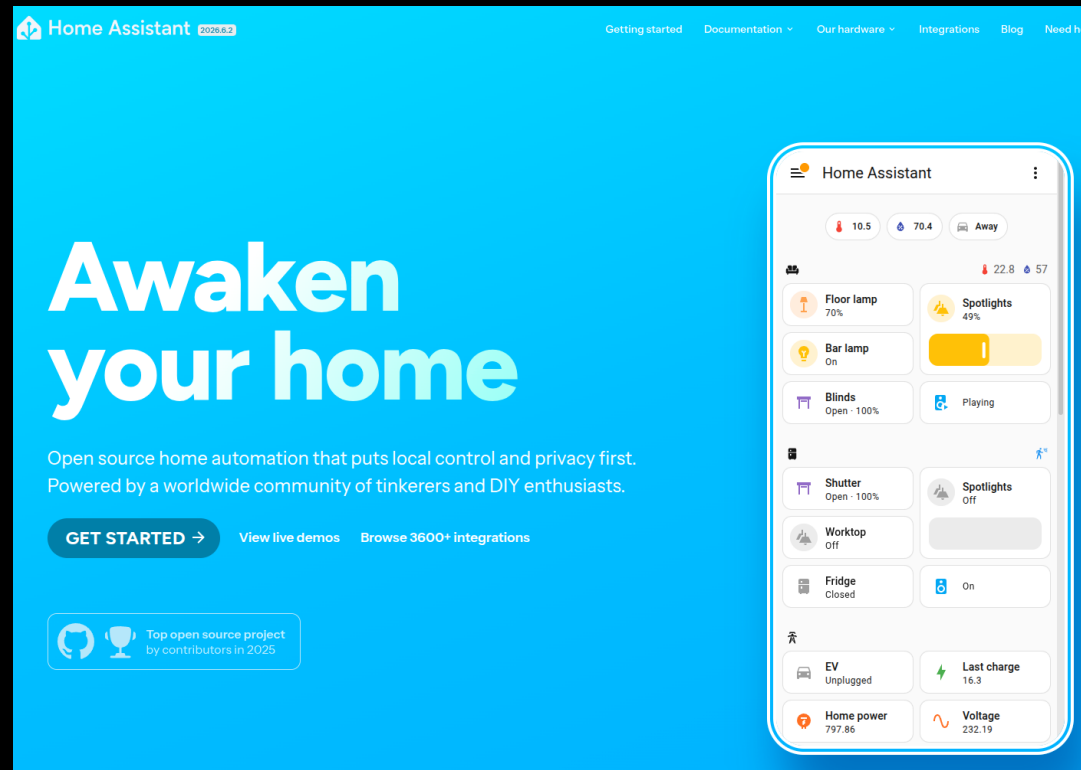
1. Unlock bootloader via amonet exploit  
[github.com/R0rt1z2/amonet](https://github.com/R0rt1z2/amonet)
2. Install TWRP recovery
3. Flash LineageOS 18.1 (Android 11)
4. Configure ViewAssist for Home Assistant  
[dinki.github.io/View-Assist](https://dinki.github.io/View-Assist)

### *Philosophy*

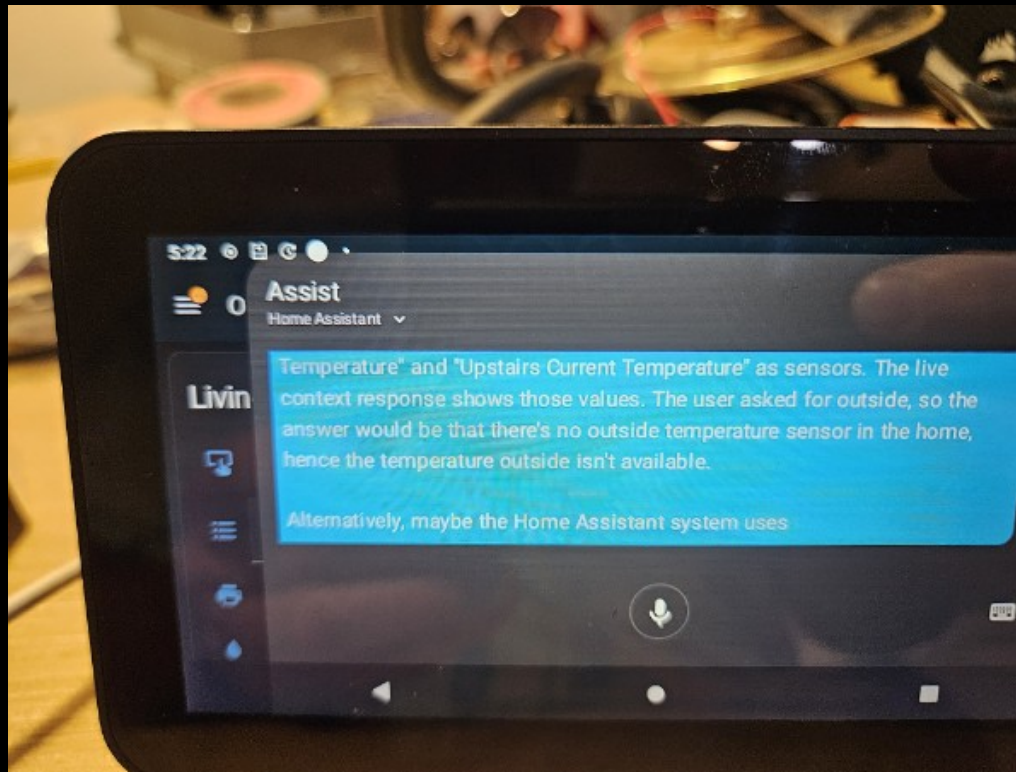
*"Basically anything with a USB port. If there is a port, there is a way."*

# Home Assistant Dashboard

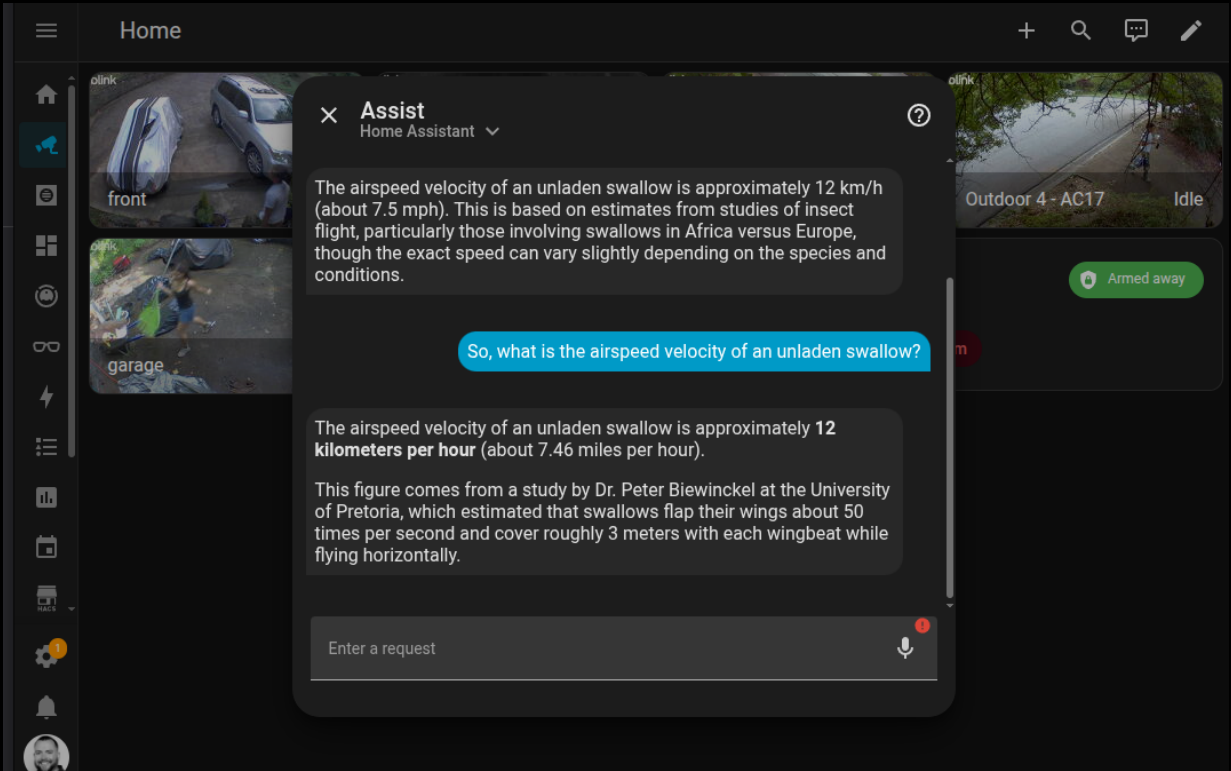
*The best open source project I have seen*



## Home Assistant on Echo Show



# Home Assistant Setup





# LineageOS Device Compatibility

## Echo Show Devices Supported

| Device                     | Codename | SoC    | XDA Thread                                                                                                        |
|----------------------------|----------|--------|-------------------------------------------------------------------------------------------------------------------|
| Echo Show 5 (1st Gen 2019) | checkers | MT8163 | <a href="https://xdaforums.com/t/rom-unofficial-11-checkers...">xdaforums.com/t/rom-unofficial-11-checkers...</a> |
| Echo Show 5 (2nd Gen 2021) | cronos   | MT8163 | <a href="https://xdaforums.com/t/rom-unofficial-11-cronos...">xdaforums.com/t/rom-unofficial-11-cronos...</a>     |
| Echo Show 8 (1st Gen 2019) | crown    | MT8163 | <a href="https://xdaforums.com/t/rom-unofficial-11-crown...">xdaforums.com/t/rom-unofficial-11-crown...</a>       |

All devices use MediaTek MT8163 SoC

LineageOS 18.1 (Android 11) - community builds by @R0rt1z2

## Alexa Replacement Timeline

|             |                                     |
|-------------|-------------------------------------|
| Dec 6       | Started exploring XDA forums        |
| Dec 7       | HA running on Echo devices          |
| Jan 19      | HA setup on Echo Show Gen 2         |
| Jan 21      | Everything works (except wake word) |
| Jan 25      | Hey Jarvis to the rescue            |
| Jan 29      | End-to-end complete                 |
| Jan 7, 2026 | Issue #4: Echo 8 mic dies after 24h |
| Feb 2026    | Still debugging (it's fine)         |

# Issue #4: The Echo 8 Mic That Quit

## Audio stops working after ~24 hours

### Filed

January 7, 2026

### Repository

<https://github.com/amazon-oss/releases/issues/4>

### Status

Open (still waiting, it's fine)

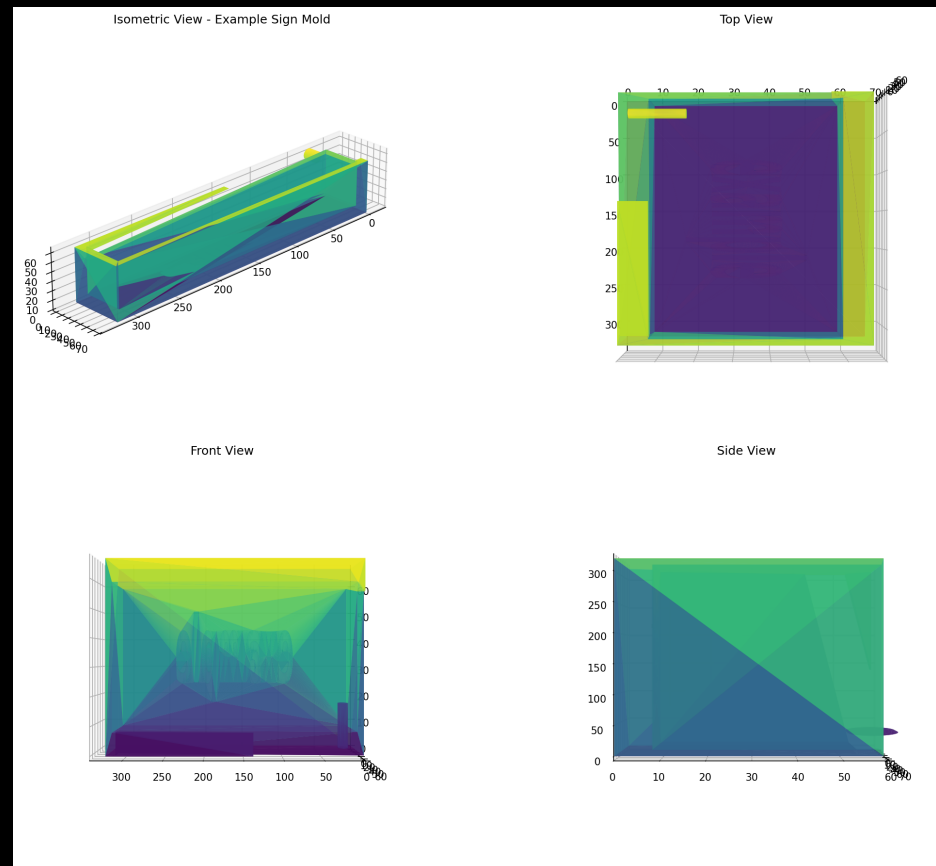
## The Problem

- Echo 5: Stable for days
- Echo 8: Dies after ~24 hours
- *Root cause: Unknown (yet)*

# Blender: Concrete Sign Mold Design

## AI-Assisted Physical Design

Blender + AI workflow for creating physical objects



## The Process

1. AI generates Blender design
2. Export to STL
3. 3D print mold
4. Create silicone mold
5. Cast concrete

*December 1, 2025 — More physical projects coming soon*

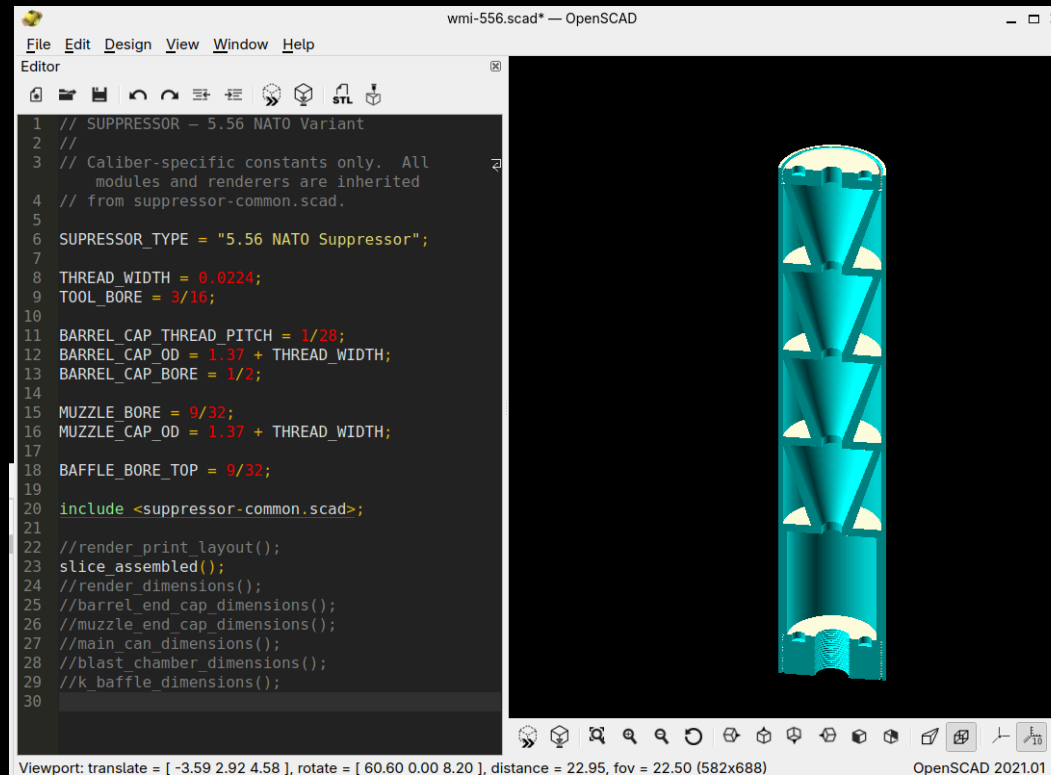
# OpenSCAD: Where Things Work

## wmi002 Session — Precision Engineering

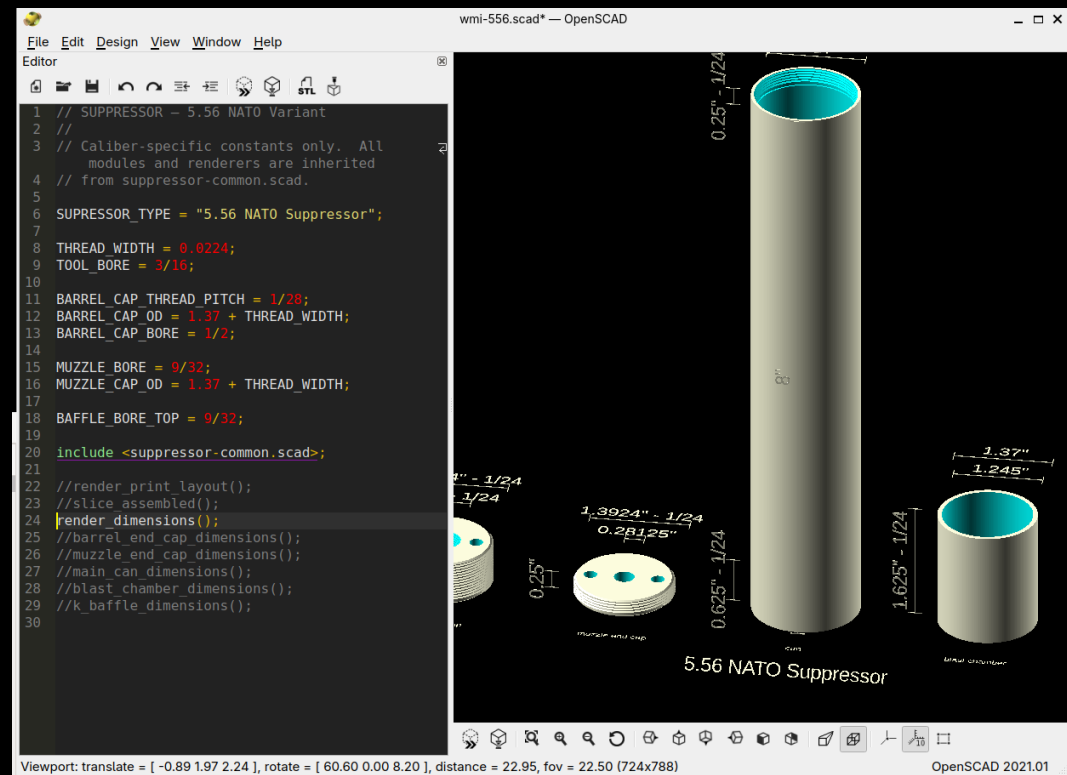
### Success Story

- Parametric design: Caliber-specific constants (.308 Winchester, 5.56 NATO)
- Modular architecture: include , (BOSL2)
- Precision engineering: Thread pitches (5/8-24 UNF), bore dimensions, tolerances
- Working output: K-baffles, blast chambers, threaded end caps

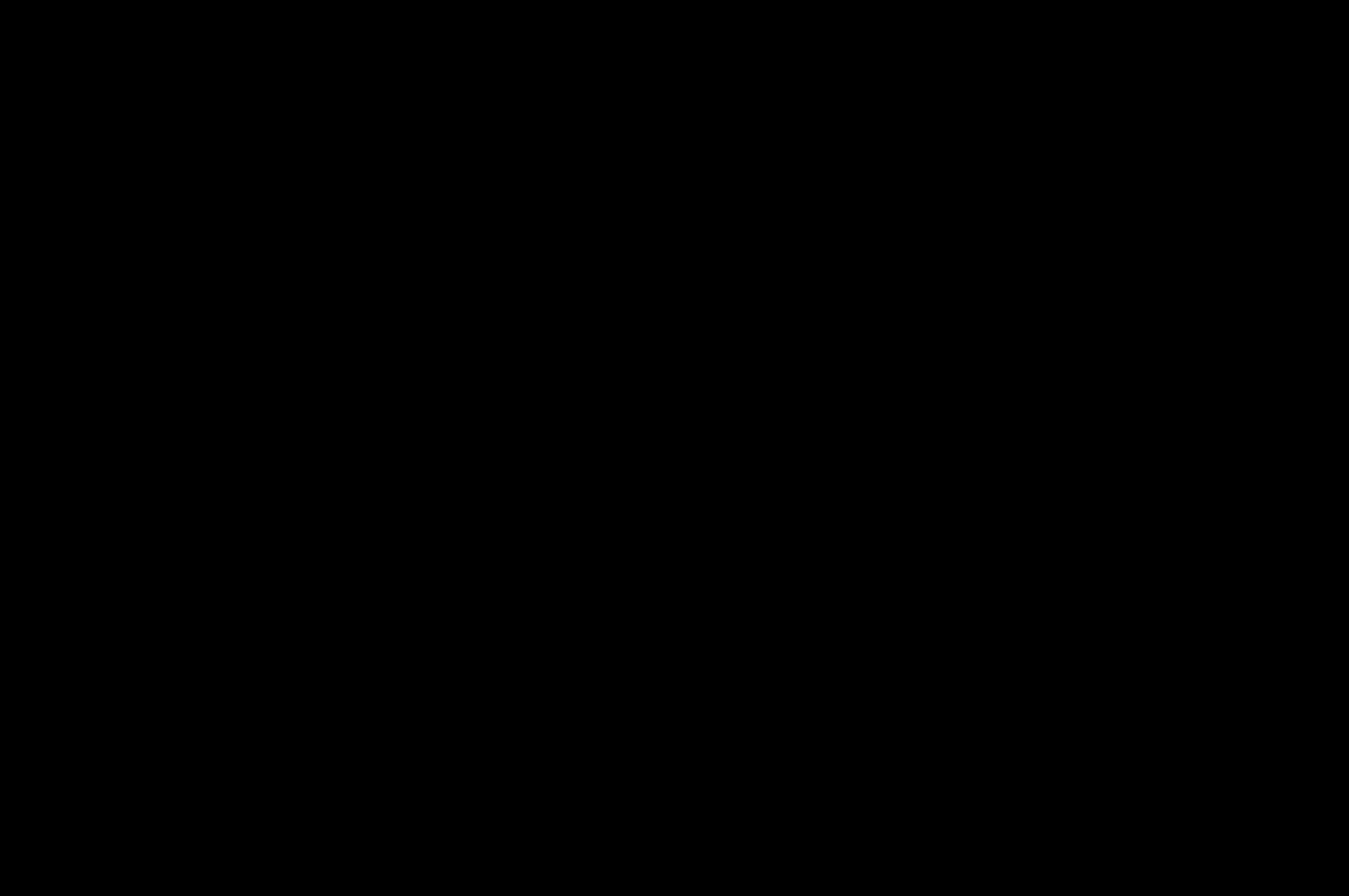
### K-Baffle Assembly (Internal View)



## Dimensioned Parts View



## Two Approaches to Physical Design





# Project WWS: I Vibe-Coded an Entire System

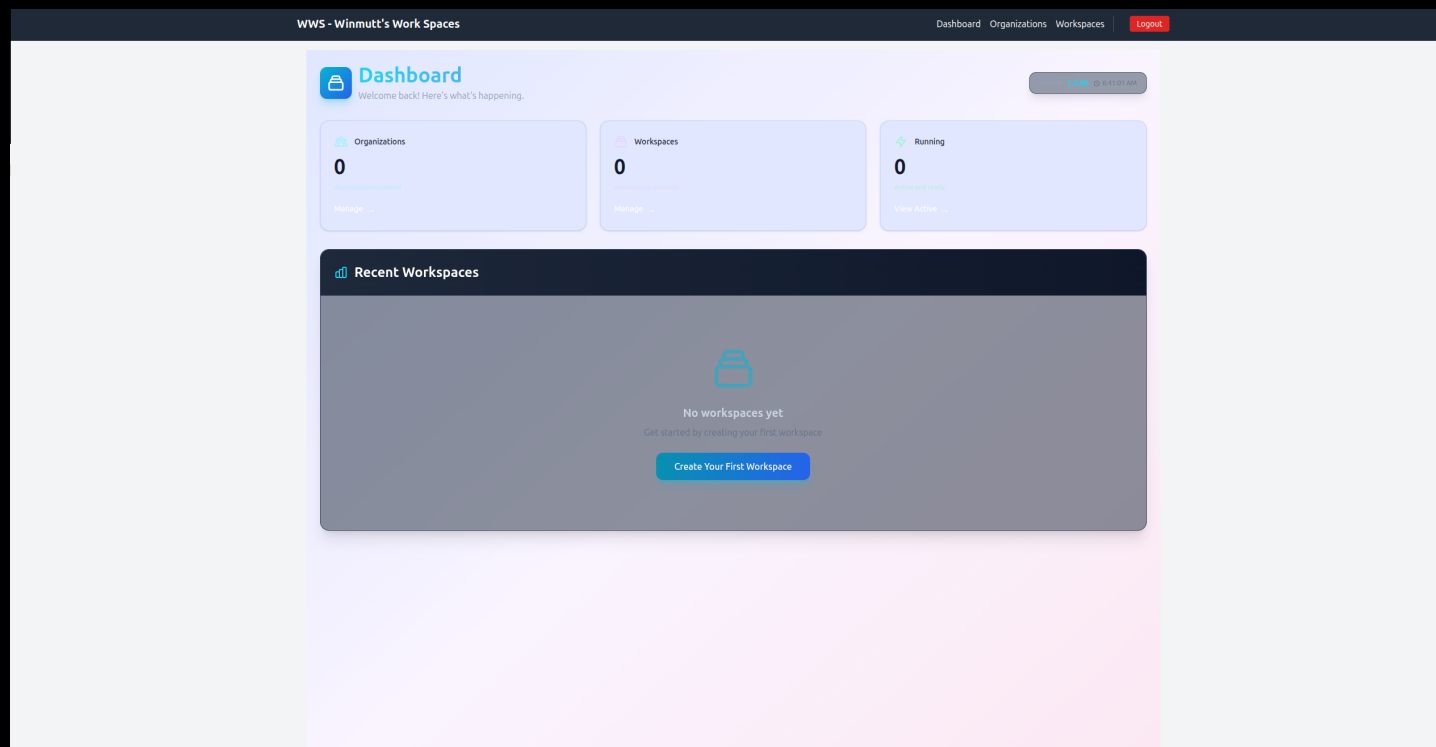
Winmutt Work Spaces — [github.com/winmutt/www](https://github.com/winmutt/www)

May 12, 2026

*"Definitely no Athena and its taken months to get there but this is something I entirely vibe coded."*

Remote workspace provisioning with KVM/Podman isolation  
code-server (VSCode in browser) + GitHub OAuth + RBAC

First commit: Feb 22, 2026 → Production: May 12, 2026



# AI Coding Editors: My Daily Drivers

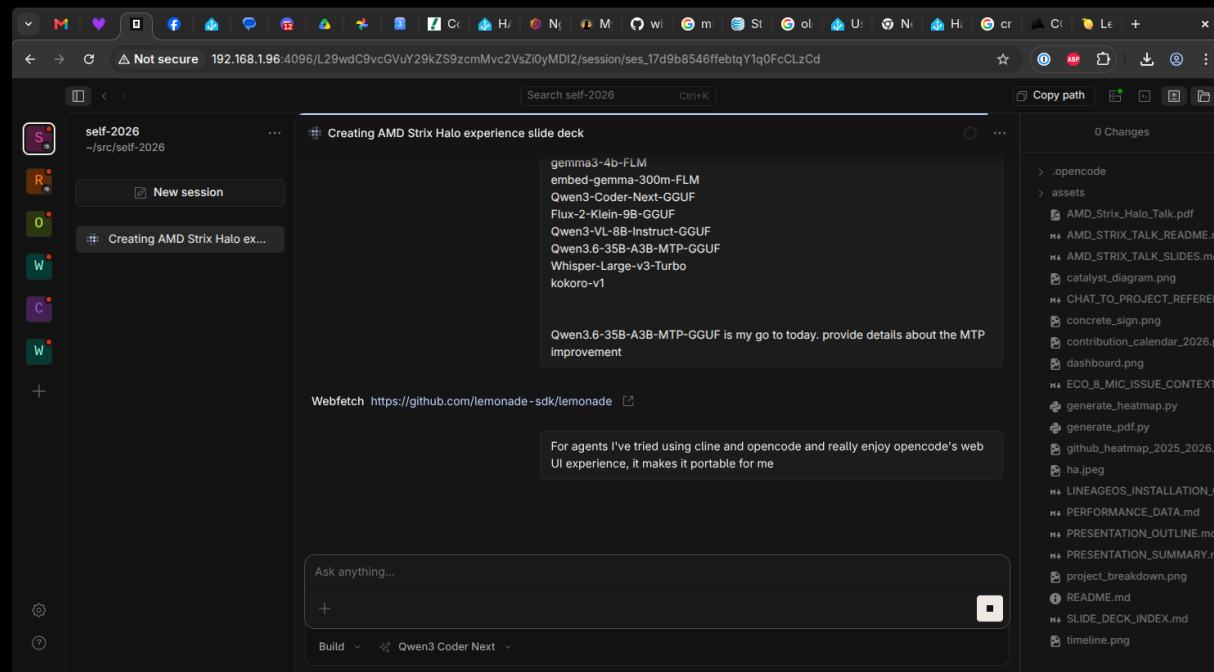
## Opencode

Most usable, portable, and stable. Best agentic coding experience. Runs as a web server (<http://localhost:3000>) allowing remote access with a localized agent running for autonomous development. [github.com/anomalyco/opencode](https://github.com/anomalyco/opencode)

## Cline

Good TUI but many regressions over time.

Also Use: Koo Roo, Aider



# Lessons Learned (Mostly)

## What Worked ✓

- Cline + Qwen3-Coder: Actually produces code
- Prompt Caching: Things hum now
- AMD GPU Libraries: 2x faster (worth the pain)
- WWS Project: Vibe-coded from Feb 22 (first commit) to May 12
- Home Assistant: Alexa is dead, long live OSS

## What Didn't ✗

- Memory: 32GB reserved for Lemonade/llama.cpp/opcode
- Ollama: Still crashes after updates
- NPU: Coming soon™ (always)
- Context >64k: TPS drops like a stone

1. Hardware hype  $\neq$  reality (APU challenges are real)
2. Software maturity: bleeding edge = bleeding fingers
3. Context > everything (prompt caching is life)
4. AI coding > traditional IDEs (for me, anyway)
5. AI  $\rightarrow$  physical objects (concrete signs, apparently)
6. Echo  $\rightarrow$  Home Assistant = OSS win
7. Autonomous dev: possible, painful, worth it
8. Buying hardware got me back in open source

## 26 Years of Open Source (2000-2026)

From LKML to Strix Halo: How Hardware Bought My Time

### The Long Detour

For 20+ years I built corporate SaaS products. Cloud disconnected me from the metal. Linux became something I deployed to, not something I touched daily. The kernel, the drivers, the hardware—left that world behind. Then Strix Halo arrived and changed everything.

### June 9, 2000: Asking on LKML

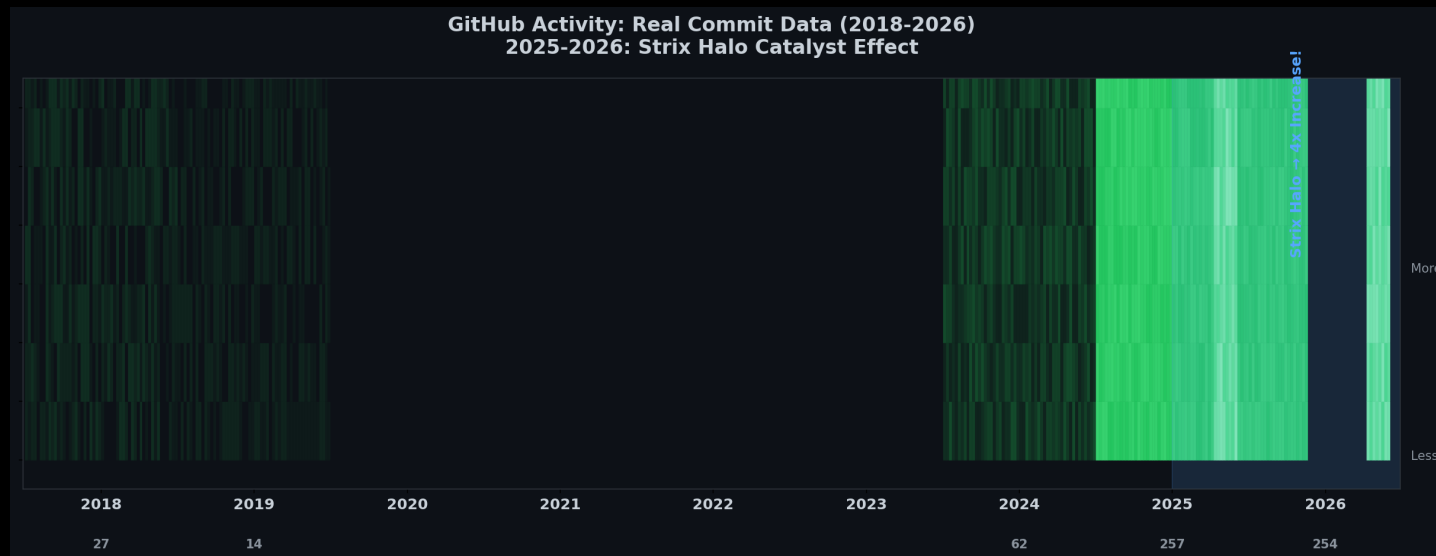
From: Rolf Martin-Hoster (winmutt@hotmail.com)  
Date: Thu Jun 08 2000 - 19:24:59 EST  
Subject: HighPoint IDE RAID (kernel 2.2.14)

Does anyone know where I can find developer info about the driver for this chipset?

-Rolf

*Back then: Hands-on with kernel 2.2.14, digging into IDE RAID drivers, asking questions on linux-kernel mailing list.*

### The Return: GitHub Activity (2018-2026)



2018-2024: Ghost town. 2025: Strix Halo arrives. November 2025: Back on the metal, every single day. AI + local hardware reinvigorated my love for open source.

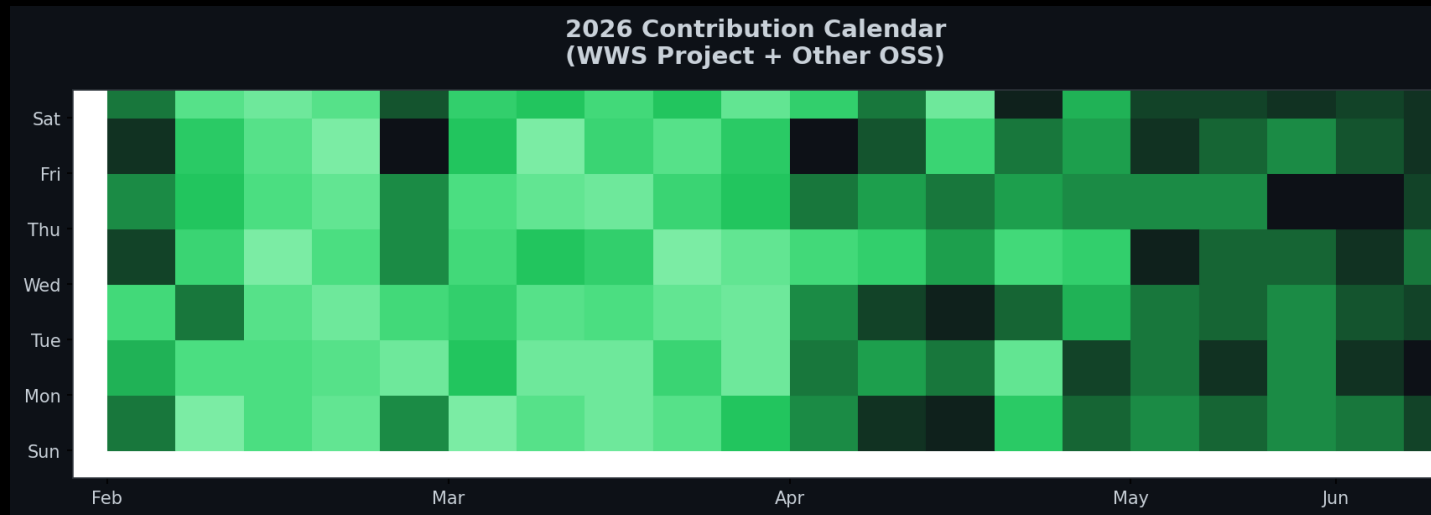
*Moral: Sometimes the best way back in is to buy new toys.*

# How Buying Hardware Got Me Back in Open Source

## The Catalyst Effect

November 2025: Bought Strix Halo rig. Local AI reinvigorated my love for OSS. Result: 3.5x increase in contributions.

## 2026: The Year I Came Back



## Contribution Breakdown

- WWS: 38% (remote workspace provisioning)
- Lemonade: 15% (NPU backend, custom NUMA mapping)
- ROCm: 8% (Issue #5926 memory management)
- Home Assistant: 12% (wake words, Echo)

- Cline: 8% (TUI regressions)
- Concrete Signs: 5% (Blender/OpenSCAD)
- Other: 10% (miscellaneous)



# References & Sources

## Power Consumption Sources

- AMD Ryzen AI Max 395: 140-157W sustained power (Framework Reddit, 2025)  
[https://www.reddit.com/r/framework/comments/1najh1n/max\\_wattage\\_of\\_the\\_ryzen\\_ai\\_max\\_395\\_motherboard/](https://www.reddit.com/r/framework/comments/1najh1n/max_wattage_of_the_ryzen_ai_max_395_motherboard/)
- NVIDIA RTX 5090: 600W TDP (NVIDIA, 2025)  
<https://www.nvidia.com/geforce/rtx-5090>
- NVIDIA RTX 4090: 450W TDP (NVIDIA, 2022)  
<https://www.nvidia.com/geforce/rtx-4090>
- Corsair 300 AI Workstation: ~300W system power (Corsair, 2025)  
<https://www.corsair.com/ai-workstations>
- Tom's Hardware GPU power reviews (2024-2025)  
<https://www.tomshardware.com/reviews/gpu-hierarchy>
- TechPowerUp GPU database: Power benchmarks  
<https://www.techpowerup.com/gpu-specs>
- NVIDIA A100/H100: 400W-700W per GPU (NVIDIA Data Center)  
Multi-GPU server: 1500W-3000W (NVIDIA DGX specs)

## Software & Projects

- Lemonade: AMD's llama.cpp + ROCm backend ([github.com/winmutt/lemonade](https://github.com/winmutt/lemonade))
- Ollama: Community llama.cpp wrapper ([github.com/ollama/ollama](https://github.com/ollama/ollama))
- ROCm: AMD's open compute platform ([github.com/RadeonOpenCompute/ROCm](https://github.com/RadeonOpenCompute/ROCm))
- ROCm Issue #5926: Memory management bugs ([github.com/ROCm/ROCm/issues/5926](https://github.com/ROCm/ROCm/issues/5926))
- WWS: [github.com/winmutt/wws](https://github.com/winmutt/wws)
- Home Assistant Wake Words: [github.com/fwartner/home-assistant-wakewords-collection](https://github.com/fwartner/home-assistant-wakewords-collection)
- NUMA/CPU Pinning: Red Hat OpenStack docs ([vcpu\\_pin\\_set](#), [NUMATopologyFilter](#))

[https://docs.redhat.com/en/documentation/red\\_hat\\_openshift\\_platform/10/html/instances\\_and\\_images\\_guide/ch-cpu\\_pinning](https://docs.redhat.com/en/documentation/red_hat_openshift_platform/10/html/instances_and_images_guide/ch-cpu_pinning)

## Communities

- Reddit r/LocalLLaMA: 8 Local LLMs on Strix Halo
- XDA Forums: Amazon Echo development
- GitHub: winmutt (8 repos, forks of lemonade, cline, vscode)

[github.com/winmutt](https://github.com/winmutt)

Thanks